

LeetCode 2333 — Minimum Sum of Squared Difference

Language: Java

Question

You are given two integer arrays **nums1** and **nums2** of equal length, and two integers **k1** and **k2**. You may increase or decrease elements of **nums1** at most **k1** times and elements of **nums2** at most **k2** times. Return the minimum possible sum of squared differences between corresponding elements.

Example

Input: nums1 = [1,4,10,12], nums2 = [5,8,6,9], k1 = 1, k2 = 1

Output: 43

Java Solution

```
import java.util.Arrays;

class Solution {
    public long minSumSquareDiff(int[] nums1, int[] nums2, int k1, int k2) {
        int n = nums1.length;
        long[] diff = new long[n];
        long totalDiff = 0;
        long maxDiff = 0;

        for (int i = 0; i < n; i++) {
            diff[i] = Math.abs((long) nums1[i] - nums2[i]);
            totalDiff += diff[i];
            maxDiff = Math.max(maxDiff, diff[i]);
        }

        long k = (long) k1 + k2;
        if (k >= totalDiff) {
            return 0;
        }

        long low = 0, high = maxDiff;

        // Find the smallest level where the required reductions <= k
        while (low < high) {
            long mid = low + (high - low) / 2;
            long needed = 0;

            for (long d : diff) {
                if (d > mid) {
                    needed += d - mid;
                }
                if (needed > k) {
                    break;
                }
            }

            if (needed <= k) {
                high = mid;
            } else {
                low = mid + 1;
            }
        }

        long level = low;
        long used = 0;
        long answer = 0;
        long countAtOrAboveLevel = 0;

        for (long d : diff) {
            long capped = Math.min(d, level);
            answer += capped * capped;
        }
    }
}
```

```

        if (d > level) {
            used += d - level;
        }
        if (d >= level) {
            countAtOrAboveLevel++;
        }
    }

    // Spend leftover operations by reducing level-sized differences by 1
    long remaining = k - used;
    answer -= remaining * (2 * level - 1);

    return answer;
}
}

```

Simple Logic

1. Calculate the absolute difference at every index.
2. If the total number of operations needed to make every difference zero is at most $k_1 + k_2$, return 0.
3. Use binary search to find the smallest maximum difference level that can be reached within the available operations.
4. Cap each difference at that level, calculate the sum of squares, and use any remaining operations to reduce some differences at that level by one.

Complexity

Time: $O(n \log M)$, where M is the largest absolute difference.

Extra space: $O(n)$, for the difference array.